LAB EXPERIMENTS

Q23) Write a C program for Encrypt and decrypt in counter mode using one of the following ciphers: affine modulo 256, Hill modulo 256, S-DES. Test data for S-DES using a counter starting at 0000 0000. A binary plaintext of 0000 0001 0000 0010 0000 0100 encrypted with a binary key of 01111 11101 should give a binary plaintext of 0011 1000 0100 1111 0011 0010. Decryption should work correspondingly.

Aim:

To write a C program to implement **Counter (CTR) Mode** encryption and decryption using the **Affine Cipher modulo 256**.

Algorithm:

1. Start the program.
2. Read the plaintext from the user.
3. Read the multiplicative key (**a**), additive key (**b**), and initial counter value.
4. Verify that **a** is coprime with 256.
5. Encrypt the counter using the Affine Cipher:
   $E=(a\times Counter+b) mod 256$
6. XOR the encrypted counter with the plaintext byte to produce the ciphertext byte.
7. Increment the counter for the next plaintext byte.
8. Repeat until all plaintext bytes are encrypted.

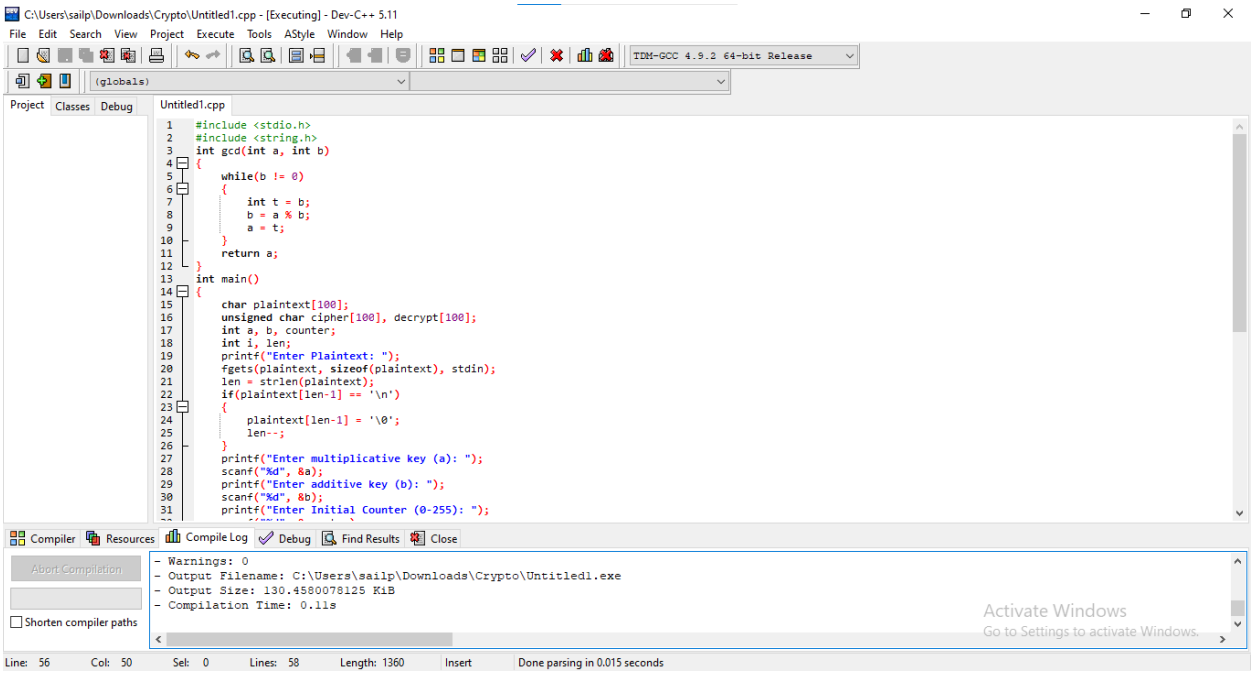
Decryption

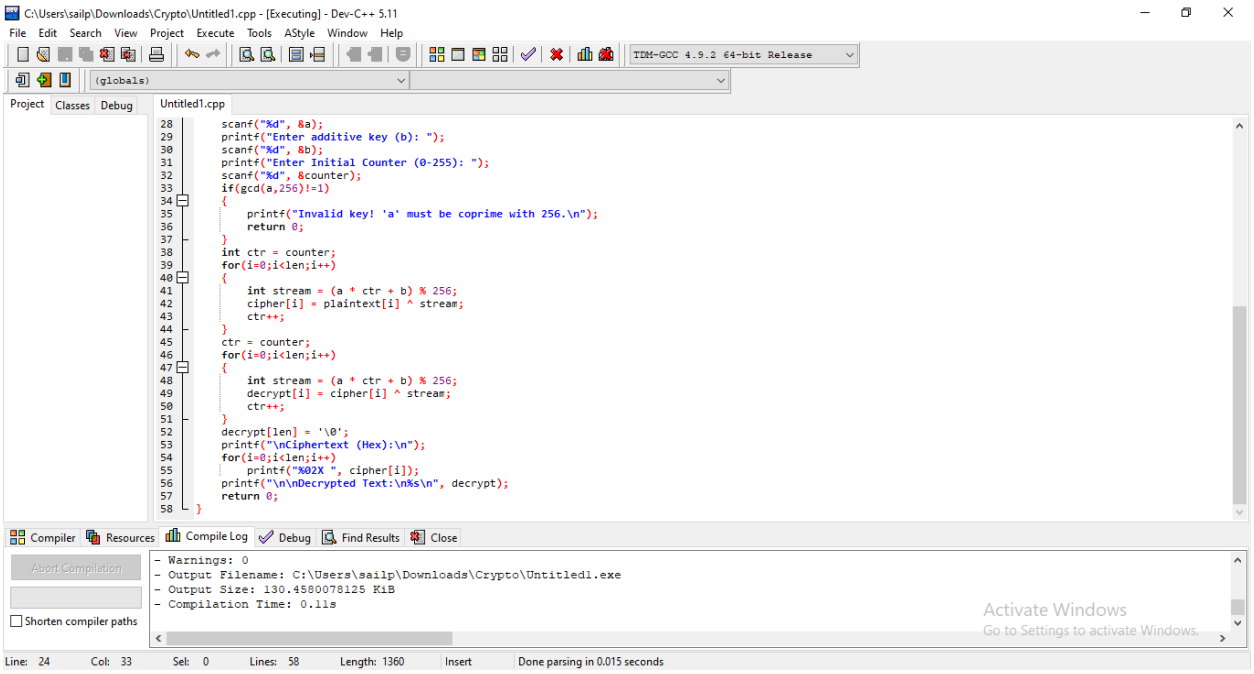
9. Reset the counter to its initial value.
10. Encrypt the counter again using the same Affine Cipher.
11. XOR the encrypted counter with the ciphertext byte to recover the plaintext.
12. Display the decrypted plaintext and stop the program.

Result:

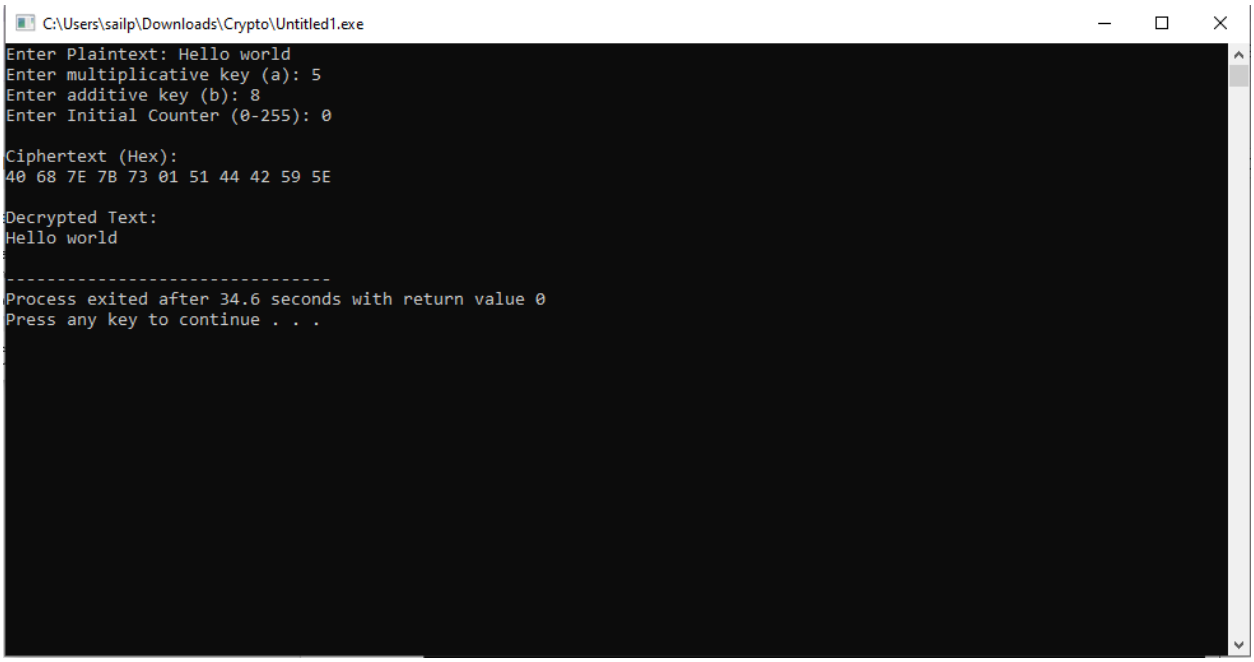
Thus, the C program to implement **Counter (CTR) Mode** using the **Affine Cipher modulo 256** was successfully executed. The plaintext was encrypted using an encrypted counter stream and successfully decrypted back to the original plaintext.

Code:


C:\Users\sailp\Downloads\Crypto\Untitled1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
Untitled1.cpp
#include <stdio.h>
#include <string.h>
int gcd(int a, int b)
{
while(b != 0)
{
int t = b;
b = a % b;
a = t;
}
return a;
}
int main()
{
char plaintext[100];
unsigned char cipher[100], decrypt[100];
int a, b, counter;
int i, len;
printf("Enter Plaintext: ");
fgets(plaintext, sizeof(plaintext), stdin);
len = strlen(plaintext);
if(plaintext[len-1] == '\n')
{
plaintext[len-1] = '\0';
len--;
}
printf("Enter multiplicative key (a): ");
scanf("%d", &a);
printf("Enter additive key (b): ");
scanf("%d", &b);
printf("Enter Initial Counter (0-255): ");
Compiler Resources Compile Log Debug Find Results Close
Abort Compilation
Shorten compiler paths
- Warnings: 0
- Output Filename: C:\Users\sailp\Downloads\Crypto\Untitled1.exe
- Output Size: 130.4580078125 KiB
- Compilation Time: 0.11s
Activate Windows
Go to Settings to activate Windows.
Line: 56 Col: 50 Sel: 0 Lines: 58 Length: 1360 Insert Done parsing in 0.015 seconds


C:\Users\sailp\Downloads\Crypto\Untitled1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
Untitled1.cpp
scanf("%d", &a);
printf("Enter additive key (b): ");
scanf("%d", &b);
printf("Enter Initial Counter (0-255): ");
scanf("%d", &counter);
if(gcd(a,256)!=1)
{
printf("Invalid key! 'a' must be coprime with 256.\n");
return 0;
}
int ctr = counter;
for(i=0;i<len;i++)
{
int stream = (a * ctr + b) % 256;
cipher[i] = plaintext[i] ^ stream;
ctr++;
}
ctr = counter;
for(i=0;i<len;i++)
{
int stream = (a * ctr + b) % 256;
decrypt[i] = cipher[i] ^ stream;
ctr++;
}
decrypt[len] = '\0';
printf("\nCiphertext (Hex):\n");
for(i=0;i<len;i++)
printf("%02X ", cipher[i]);
printf("\n\nDecrypted Text:\n%s\n", decrypt);
return 0;
}
Compiler Resources Compile Log Debug Find Results Close
Abort Compilation
Shorten compiler paths
- Warnings: 0
- Output Filename: C:\Users\sailp\Downloads\Crypto\Untitled1.exe
- Output Size: 130.4580078125 KiB
- Compilation Time: 0.11s
Activate Windows
Go to Settings to activate Windows.
Line: 24 Col: 33 Sel: 0 Lines: 58 Length: 1360 Insert Done parsing in 0.015 seconds

Output:


C:\Users\sailp\Downloads\Crypto\Untitled1.exe
Enter Plaintext: Hello world
Enter multiplicative key (a): 5
Enter additive key (b): 8
Enter Initial Counter (0-255): 0
Ciphertext (Hex):
40 68 7E 7B 73 01 51 44 42 59 5E
Decrypted Text:
Hello world

Process exited after 34.6 seconds with return value 0
Press any key to continue . . .